struct

Description	accesso a struct, come memorizzare struct in array

`struct`

Una struttura dati simil oggetto

```
struct Persona {
  // code
};

int main() {
  struct Persona persona1, persona2, p[20];
  return 0;
}
```

```
struct Persona {
   // code
} persona1, persona2, p[20];
```

In entrambi `persona1, persona2, struct Persona;` sono variabili

Accesso a elementi

```
persona2.salario
```

Tramite `.` è possibile accedere ai vari campi della struct associati ad una variabile di tipo struct

```
pPersona->nome
```

L'operatore `->` semplifica l'accesso ai membri di una struttura attraverso un puntatore, risparmiando la necessità di dereferenziare il puntatore e poi utilizzare l'operatore `.`

> 📌 VARIABILE NON PUNTATORE `.`
> VARIABILE PUNTATORE `->`

Quindi la differenza tra queste due modalità di accesso è la variabile alla quale si utilizza.
`->` viene utilizzato quando lavoriamo su variabili puntatore

`.` viene utilizzato quando lavoriamo su variabili non puntatore

```
#include <stdio.h>

int main() {
    struct Persona unaPersona;
    struct Persona *pPersona;

    // Inizializzazione della struttura
    strcpy(unaPersona.nome, "Mario");
    unaPersona.eta = 30;

    // Il puntatore punta alla struttura
    pPersona = &unaPersona;

    // Accesso ai membri della struttura con l'operatore ->
    printf("Nome: %s\n", pPersona->nome);
    printf("Età: %d\n", pPersona->eta);

    return 0;
}
```

📌 L'operatore `->` è specificamente progettato per essere utilizzato con puntatori a strutture in linguaggio C. Serve come una comoda abbreviazione per accedere ai membri di una struttura attraverso un puntatore senza dover dereferenziare esplicitamente il puntatore.

ESEMPIO:

```
#include <stdio.h>
#include <string.h>

// create struct with person1 variable
struct Person {
  char name[50];
  int citNo;
  float salary;
} person1;

int main() {

  // assign value to name of person1
  strcpy(person1.name, "George Orwell");

  // assign values to other person1 variables
  person1.citNo = 1984;
  person1. salary = 2500;

  // print struct variables
  printf("Name: %s\n", person1.name);
  printf("Citizenship No.: %d\n", person1.citNo);
  printf("Salary: %.2f", person1.salary);

  return 0;
}
```

OUTPUT:

```
Name: George Orwell
Citizenship No.: 1984
Salary: 2500.00
```

Array di struct

(. . ., 52.874932, 12.43242)	(. . . , 53.423423, 12.432234)

Ogni elemento usa **24 byte**

Array di puntatori a struct

*	*

Ogni puntatore a una struct usa ***8 byte*** quindi questa modalità d’uso di array si struct è maggiormente usata in quanto si utilizza meno memoria.

Anche se la struct a cui si punta è gigante il suo puntatore sarà *sempre* di **8 byte**